Assignment 6(Problem Set 6 & 7)

Sayan Ganguly,Roll:714

2026-03-19

Problem Set 6: Model Selection

To demonstrate the application of subset selection methods

Consider the "Hitters" data. For loading the data you need to install ISLR library. We wish to predict a baseball player's salary on the basis of various predictors. So first we need to choose an optimal model.

```
library(ISLR)
head(Hitters)

##                    AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits
CHmRun
## -Andy Allanson      293   66     1   30  29    14     1    293    66
1
## -Alan Ashby         315   81     7   24  38    39    14   3449   835
69
## -Alvin Davis        479  130    18   66  72    76     3   1624   457
63
## -Andre Dawson       496  141    20   65  78    37    11   5628  1575
225
## -Andres Galarraga   321   87    10   39  42    30     2    396   101
12
## -Alfredo Griffin    594  169     4   74  51    35    11   4408  1133
19
##                    CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Andy Allanson        30   29     14      A        E     446      33     20
## -Alan Ashby          321  414    375      N        W     632      43     10
## -Alvin Davis         224  266    263      A        W     880      82     14
## -Andre Dawson        828  838    354      N        E     200      11      3
## -Andres Galarraga     48   46     33      N        E     805      40      4
## -Alfredo Griffin     501  336    194      A        W     282     421     25
##                    Salary NewLeague
## -Andy Allanson         NA         A
## -Alan Ashby         475.0         N
## -Alvin Davis        480.0         A
## -Andre Dawson       500.0         N
## -Andres Galarraga    91.5         N
## -Alfredo Griffin    750.0         A

set.seed(123)
```

1. Clean the data by removing all rows that have missing value in any variable.You will be working with this clean data.

```
data_clean = na.omit(Hitters)
head(data_clean)
```

```
##                    AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits
CHmRun
## -Alan Ashby          315   81     7   24  38    39    14   3449   835
69
## -Alvin Davis         479  130    18   66  72    76     3   1624   457
63
## -Andre Dawson        496  141    20   65  78    37    11   5628  1575
225
## -Andres Galarraga    321   87    10   39  42    30     2    396   101
12
## -Alfredo Griffin     594  169     4   74  51    35    11   4408  1133
19
## -Al Newman           185   37     1   23   8    21     2    214    42
1
##                    CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Alan Ashby          321  414    375      N        W     632      43     10
## -Alvin Davis         224  266    263      A        W     880      82     14
## -Andre Dawson        828  838    354      N        E     200      11      3
## -Andres Galarraga     48   46     33      N        E     805      40      4
## -Alfredo Griffin     501  336    194      A        W     282     421     25
## -Al Newman            30    9     24      N        E      76     127      7
##                    Salary NewLeague
## -Alan Ashby         475.0         N
## -Alvin Davis        480.0         A
## -Andre Dawson       500.0         N
## -Andres Galarraga    91.5         N
## -Alfredo Griffin    750.0         A
## -Al Newman           70.0         A
```

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
train_sample = sample(1:nrow(data_clean), 0.8*nrow(data_clean))
train = data_clean[train_sample, ]
test = data_clean[-train_sample, ]
```

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R squared and Cp values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```
library(leaps)
reg1 = regsubsets(Salary ~ ., data = train, nvmax = 19)
summary(reg1)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19)
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: exhaustive
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    " "
"*"
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
```

```
"*"
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```r
reg1_sum = summary(reg1)

adjr2_best = which.max(reg1_sum$adjr2)
cp_best    = which.min(reg1_sum$cp)

adjr2_best
```

```
## [1] 12
```

```r
cp_best
```

```
## [1] 11
```

So the best fitting model by adjusted R^2 and Cp measure are respectively 12th and 11th model.

Prediction Function

```
predict_regsubsets = function(object, newdata, id){
  form = as.formula(object$call[[2]])
  mat = model.matrix(form, newdata)
  coefi = coef(object, id = id)
  vars = names(coefi)
  mat[, vars] %*% coefi
}
```

MSE function

```
MSE = function(y, yhat){
  mean((y - yhat)^2)
}
```

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```
# AdjR2 model
train_pred_adjr2 = predict_regsubsets(reg1, train, adjr2_best)
test_pred_adjr2  = predict_regsubsets(reg1, test, adjr2_best)

# Cp model
train_pred_cp = predict_regsubsets(reg1, train, cp_best)
test_pred_cp  = predict_regsubsets(reg1, test, cp_best)

data.frame(
  Model = c("AdjR2","Cp"),
  Train_MSE = c(
    MSE(train$Salary, train_pred_adjr2),
    MSE(train$Salary, train_pred_cp)
  ),
  Test_MSE = c(
    MSE(test$Salary, test_pred_adjr2),
    MSE(test$Salary, test_pred_cp)
  )
)

##   Model Train_MSE Test_MSE
## 1 AdjR2  88935.80 130066.9
## 2    Cp  89790.63 125652.3
```

5. Repeat 3-4 using forward stepwise selection method.

```
reg2 = regsubsets(Salary ~ ., data = train, nvmax = 19, method = 'forward')
summary(reg2)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19, method =
"forward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: forward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
"*"
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 10  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 11  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
```

```
## 12  ( 1 ) " "   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 13  ( 1 ) "*"   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 17  ( 1 ) "*"   "*"  " "   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  "*"    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 5  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 7  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 8  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  "*"    " "     "*"       "*"     " "     " "    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

Best model

```
reg2_sum = summary(reg2)

adjr2_f = which.max(reg2_sum$adjr2)
cp_f    = which.min(reg2_sum$cp)

adjr2_f

## [1] 14

cp_f

## [1] 11
```

MSE

```
train_pred_f_r2 = predict_regsubsets(reg2, train, adjr2_f)
test_pred_f_r2  = predict_regsubsets(reg2, test, adjr2_f)
test_pred_f_cp  = predict_regsubsets(reg2, test, cp_f)
train_pred_f_cp = predict_regsubsets(reg2, train, cp_f)

data.frame(
  Method = "Forward",
  Train_MSE = c(
    MSE(train$Salary, train_pred_f_r2),
    MSE(train$Salary, train_pred_f_cp)
  ),
  Test_MSE = c(
    MSE(test$Salary, test_pred_f_r2),
    MSE(test$Salary, test_pred_f_cp)
  )
)
```

```
##    Method Train_MSE Test_MSE
## 1 Forward  88287.08 134001.7
## 2 Forward  90888.44 145923.9
```

6. Repeat 3-4 using backward stepwise selection method.

```
reg3 = regsubsets(Salary ~ ., data = train, nvmax = 19, method = "backward")
summary(reg3)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train, nvmax = 19, method =
"backward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
```

```
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: backward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    "*"   " "    " "
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    "*"   " "    " "
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
" "
## 15  ( 1 ) "*"   "*"  " "   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 16  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
```

```
## 7  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

Best Model

```
reg3_sum = summary(reg3)

adjr2_b = which.max(reg3_sum$adjr2)
cp_b    = which.min(reg3_sum$cp)

adjr2_b

## [1] 13

cp_b

## [1] 13
```

MSE

```
train_pred_b_r2 = predict_regsubsets(reg3, train, adjr2_b)
test_pred_b_r2  = predict_regsubsets(reg3, test, adjr2_b)
train_pred_b_cp = predict_regsubsets(reg3, train, cp_b)
test_pred_b_cp = predict_regsubsets(reg3, test, cp_b)

data.frame(
  Method = "Backward",
  Train_MSE = c(
    MSE(train$Salary, train_pred_b_r2),
    MSE(train$Salary, train_pred_b_cp)
  ),
  Test_MSE = c(
    MSE(test$Salary, test_pred_b_r2),
    MSE(test$Salary, test_pred_b_cp)
  )
)

##     Method Train_MSE Test_MSE
## 1 Backward  88865.02 134736.7
## 2 Backward  88865.02 134736.7
```

7. Compare the results of the three methods in terms of choice of best model, train MSE amd test MSE.

1)Comparison of best models

The best model for best subset selection through adjusted R^2 is 12 and through Cp is 11.

The best model for forward selection through adjusted R^2 is 14 and through Cp is 11.

The best model for backward selection through adjusted R^2 and Cp both are 13.

2)Comparison of Train MSE

Forward Selection: 88287.08 (lowest)

Backward Selection: 88865.02

Best Subset (AdjR^2): 88935.80

Best Subset (Cp): 89790.63 (highest)

Hence,Forward selection method gives the lowest training MSE which is the training MSE of the model selected using adjusted R^2, meaning it fits the training data slightly better than the others.The model obtained using Cp criteria in best subset yields the highest training mse.

3)Comparison of Test MSE

Best Subset (Cp): 125652.3 (lowest)

Best Subset (AdjR^2): 130066.9

Forward Selection: 134001.7

Backward Selection: 134736.7 (highest)

Hence,the Best Subset model obtained by Cp criterion gives the lowest test MSE whereas backward selection method gives the highest test MSE.

Conclusion

1) Although forward selection has the lowest training MSE, its test MSE is higher which indicates slight overfitting.

2) Backward selection performs similarly to forward but slightly worse.

3) Best subset selection, especially using Cp, provides the best balance between bias and variance.

So,the best model overall is the Best Subset Selection model chosen using Cp criterion as it has the lowest test MSE.

This shows that:

Evaluating all possible models (best subset) can lead to better generalization.

Problem Set 7: Shrinkage Methods

Comparison of least squares, ridge regression, LASSO and Elastic Net.

You would be working with Hitters data set available in the ISLR library.You have to predict the Salary based on all available variables. First make sure that you have deleted all rows with one or more missing values.

```
library(glmnet)

## Loading required package: Matrix

## Loaded glmnet 4.1-10

data(Hitters)

# Removing missing values
Hitters = na.omit(Hitters)
```

1. Split the data set into a training set and a test set randomly keeping 100 observations in test set and remaining in the training set.

```
set.seed(123)

test_index = sample(1:nrow(Hitters), 100)

test_data = Hitters[test_index, ]
train_data = Hitters[-test_index, ]

dim(train_data)

## [1] 163  20

dim(test_data)

## [1] 100  20

x_train = model.matrix(Salary ~ ., train_data)[, -1]
y_train = train_data$Salary

x_test = model.matrix(Salary ~ ., test_data)[, -1]
y_test = test_data$Salary
```

2. Train the model using least squares and report the train MSE and test MSE.

```r
lm_model = lm(Salary ~ ., data = train_data)
summary(lm_model)

##
## Call:
## lm(formula = Salary ~ ., data = train_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -809.76 -182.13  -54.69  138.36 1837.14
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  212.67449  129.87001   1.638 0.103705
## AtBat         -2.04895    0.87954  -2.330 0.021229 *
## Hits           5.85495    3.15579   1.855 0.065612 .
## HmRun          5.43622    9.22155   0.590 0.556448
## Runs          -2.49915    4.03131  -0.620 0.536288
## RBI            2.32756    3.84573   0.605 0.545984
## Walks          6.40986    2.41415   2.655 0.008827 **
## Years         -8.30051   17.14327  -0.484 0.628996
## CAtBat         0.01569    0.19354   0.081 0.935512
## CHits         -0.54632    1.02039  -0.535 0.593204
## CHmRun        -2.38189    2.49628  -0.954 0.341604
## CRuns          1.56320    1.05024   1.488 0.138840
## CRBI           1.26993    1.02979   1.233 0.219524
## CWalks        -0.95946    0.45825  -2.094 0.038049 *
## LeagueN       44.32184   98.13766   0.452 0.652220
## DivisionW   -149.54532   56.87996  -2.629 0.009496 **
## PutOuts        0.41589    0.11170   3.723 0.000282 ***
## Assists        0.43589    0.28787   1.514 0.132176
## Errors        -5.50162    5.83866  -0.942 0.347642
## NewLeagueN   -15.79562   97.46148  -0.162 0.871479
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 346.5 on 143 degrees of freedom
## Multiple R-squared:  0.4782, Adjusted R-squared:  0.4088
## F-statistic: 6.897 on 19 and 143 DF,  p-value: 1.144e-12

# Predictions
train_pred_lm = predict(lm_model, newdata = train_data)
test_pred_lm  = predict(lm_model, newdata = test_data)

# MSE function
mse = function(y, yhat){
  mean((y - yhat)^2, na.rm = TRUE)
}

# Compute MSE
```

```
train_mse_lm = mse(train_data$Salary, train_pred_lm)
test_mse_lm  = mse(test_data$Salary, test_pred_lm)

train_mse_lm

## [1] 105334.2

test_mse_lm

## [1] 87803.91
```

3. Fit ridge regression models on the training set, with grid values of λ.Report the following:

(i) the dimension of the coefficient matrix so obtained.

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. Comment on the output.

(iii) Test MSE corresponding to these two choices of λ.

```
lambda_grid = 10^seq(10, -2, length = 100)

model_ridge = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
```

i) Dimension of Coefficient Matrix

```
dim(coef(model_ridge))

## [1]  20 100
```

ii) The L2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ.

```
coef_50 = coef(model_ridge)[, 50]
coef_60 = coef(model_ridge)[, 60]

coef_50

##   (Intercept)         AtBat          Hits         HmRun          Runs
## 438.509969440   0.029684297   0.110523230   0.560867468   0.178502584
##           RBI         Walks         Years        CAtBat         CHits
##   0.252163513   0.238337160   1.002466622   0.002796464   0.010194612
##        CHmRun         CRuns          CRBI        CWalks       LeagueN
##   0.077846378   0.020089803   0.021957346   0.021866228   0.432252998
##     DivisionW       PutOuts       Assists        Errors    NewLeagueN
##  -6.916280599   0.018294166   0.002998934  -0.082552703   0.741567242

coef_60

##  (Intercept)        AtBat         Hits        HmRun         Runs
RBI
## 105.63996577   0.08310078   0.51246876   2.09635144   0.60993278
```

```
1.14623958
##        Walks        Years       CAtBat        CHits       CHmRun
CRuns
##   1.01677160   2.77366963   0.01043635   0.04276159   0.27032634
0.08012067
##         CRBI       CWalks      LeagueN    DivisionW      PutOuts
Assists
##   0.09127173   0.05840151  18.35350369 -62.46410535   0.14784491
0.04002532
##       Errors   NewLeagueN
##  -0.86723778  12.21693330
```

```
l2_norm = function(beta){
  sqrt(sum(beta^2))
}

l2_50 = l2_norm(coef_50)
l2_60 = l2_norm(coef_60)

l2_50
```

```
## [1] 438.5671
```

```
l2_60
```

```
## [1] 124.7542
```

(iii) Test MSE for 50th and 60th λ.

```
# Predictions
ridge_pred_50 = predict(model_ridge, s = lambda_grid[50], newx = x_test)
ridge_pred_60 = predict(model_ridge, s = lambda_grid[60], newx = x_test)

# Test MSE
mse_50 = mse(y_test, ridge_pred_50)
mse_60 = mse(y_test, ridge_pred_60)

mse_50
```

```
## [1] 167303.1
```

```
mse_60
```

```
## [1] 97935.56
```

Least Squares:

Fits the data without shrinkage.

May lead to overfitting.

Ridge Regression:

Shrinks coefficients towards zero using λ.

Higher λ → smaller coefficients → higher bias, lower variance.

The L2 norm decreases as λ increases:

The 50th λ has larger coefficients than the 60th λ.

Test MSE helps decide which λ gives better generalization.

Conclusion:

1. Ridge regression improves model stability by reducing variance.
2. Optimal λ is the one that minimizes test MSE.
3. Compared to least squares, ridge often performs better on test data.